Analysis and Design of Algorithms

NUS National University of Singapore

Algorithms

CS3230

Tutorial

Week 9

Question 1

PARTITION: Given an array of positive integers $A[1..n]$, can the elements in the array be partitioned into two sets of equal total sum? Consider the following algorithm, which answers the question of whether a subset of $A[1..k]$ that sums to `total` exists.

```
Partition(A,k,total)

if total = 0 then return true

if k = 0 then return false

return (Partition(A,k-1,total) or Partition(A,k-1,total-A[k]))
```

To answer the question of whether the set can be partitioned into two equal subsets, we can first sum the elements in A as S and then run `Partition(A,n,S/2)` if S is even, and return `false` if S is odd.

What is the runtime of the algorithm?

- $\Theta(n^2)$
- $\Theta(n)$
- $\Theta(S)$
- $\Theta(Sn)$
- $\Theta(2^n)$

Question 1

Answer: E $\Theta(2^n)$

First look at correctness (***optimal substructure***). Assume an optimal solution consisting of a subset *U* that sums to *total*.

- If the solution contains $A[n]$, then the set $U-\{A[n]\}$ must sum to $total-A[n]$.
- If the solution does not contain $A[n]$ then the set $U-\{A[n]\}$ must sum to *total*.

Hence (Partition(A,n-1,*total*) **or** Partition(A,n-1,$total-A[n]$)) returns correctly.

Question 1

Answer: E $\Theta(2^n)$

The recursive algorithm computes whether there is a subset of the first *k* numbers that sums to *total*.

Consider the case where the set *cannot* be partitioned equally into two sets -- *total* is never 0. Can ignore *total* in runtime computation. Recursion tree is complete tree of height *n*, or

$T(n) = 2T(n-1) + 1 = 4T(n-1) + 2 + 1 = \ldots = 2^n T(0) + 2^{n-1} + \ldots + 1 = \Theta(2^n)$

Question 2

PARTITION: Given an array of positive integers $A[1..n]$, can the elements in the array be partitioned into two sets of equal total sum? Consider the following algorithm, which answers the question of whether a subset of $A[1..k]$ that sums to `total` exists.

We first sum the elements in A as S. If S is odd we return false. Otherwise, we initialize another array $M[1..n][0..S/2]$ with *unknown* and run the following:

```
Partition(A,k,total)

if total = 0 then return true

if k = 0 or total < 0 then return false

if M[k][total] = unknown

    M[k][total]← (Partition(A,k-1,total) or Partition(A,k-1,total-A[k]))

return M[k][total]
```

What is the runtime of the algorithm?

- $\Theta(n)$
- $\Theta(S)$
- $\Theta(Sn)$
- $\Theta(2^n)$

Question 2

Answer: C $\Theta(Sn)$

Note: This is a **memoization** algorithm.

For memoization, the runtime is usually proportional to the number of subproblems times the amount of work in each subproblem.

Question 2

Answer: C $\Theta(Sn)$

We use $\Theta(Sn)$ time to initialise the matrix.
There are $\leq Sn$ subproblems and the amount of work in each subproblem is $O(1)$.
Hence total runtime is $\Theta(Sn)$. Not $\Theta(2^n)$ because of ***overlapping subproblems***.

For memoization, the runtime is usually proportional to the number of subproblems times the amount of work in each subproblem.

Question 3

PARTITION: Given an array of positive integers $A[1..n]$, can the elements in the array be partitioned into two sets of equal total sum? Consider the following algorithm:

We first sum the elements in A as S. If S is odd we return false. Otherwise, we create another array $M[0..n][0..S/2]$, initialize $M[0][0]$ to **true** and $M[0][j]$ to **false** for $j = 1, \ldots, total$, and run the following:

```
Partition(A,total)

FIRST FOR LOOP

    SECOND FOR LOOP

        if (j ≥ A[i]) then

            M[i][j] ← M[i-1][j] or M[i-1][j-A[i]]

        else

            M[i][j] ← M[i-1][j]
```

Which of the following options for

```
FIRST FOR LOOP

    SECOND FOR LOOP
```

will give correct algorithms?

[**Hint:** Draw a dependency graph - what does **M[i][j]** depend on?]

```
(I)   for i ← 1 to n
          for j ← 0 to total
(II)  for i ← 1 to n
          for j ← total downto 0
(III) for j ← 0 to total
          for i ← 1 to n
(IV)  for j ← total downto 0
          for i ← 1 to n
```

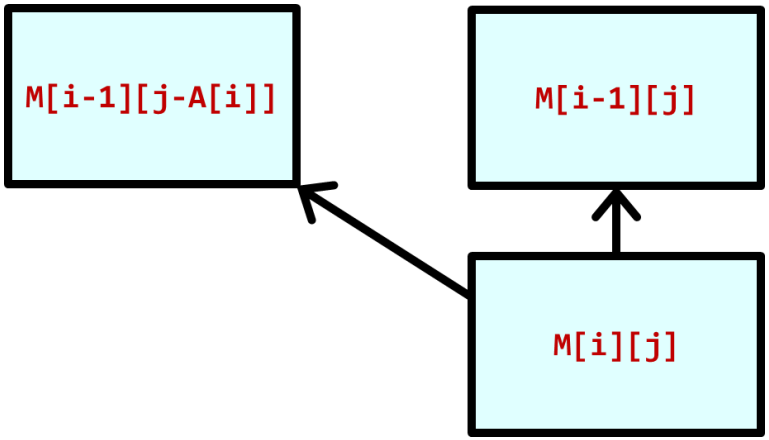
- (I) only
- (I) and (II) only
- (I), (II) and (III) only
- (I), (II), (III), and (IV)

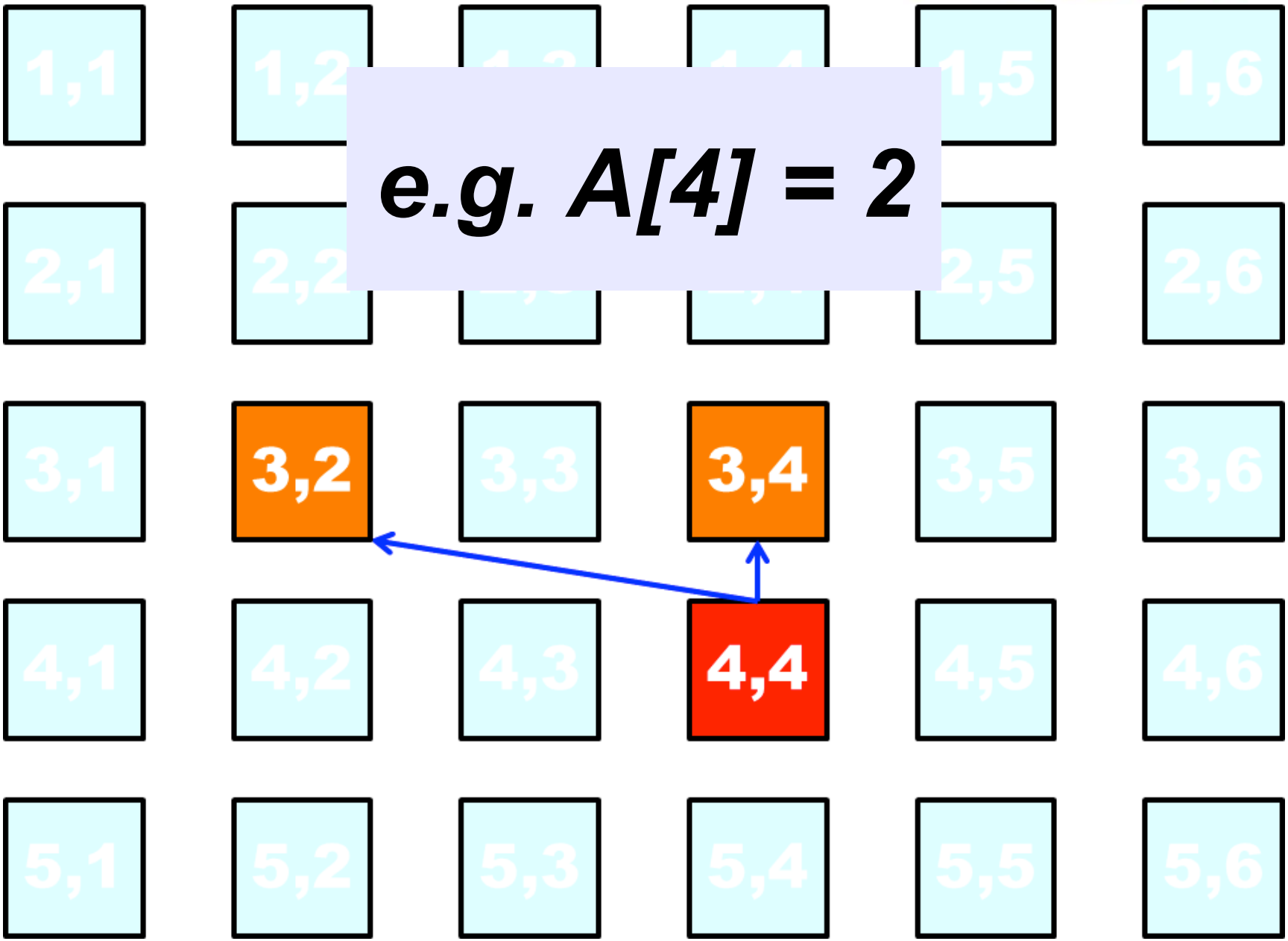
Question 3

Answer: C

This is the dynamic programming version of the algorithm -- previously recursion and memoization versions.

Ordering important in DP: ***table entries must have been computed before being used***.

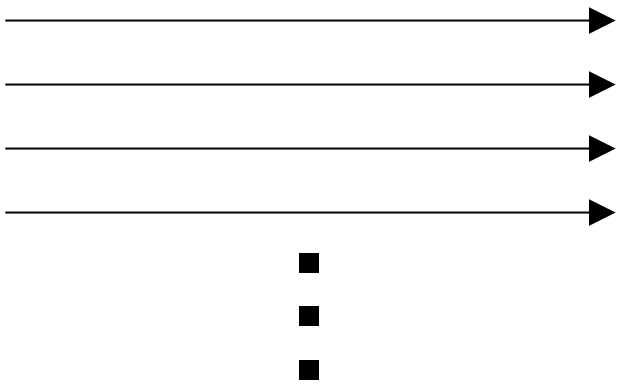


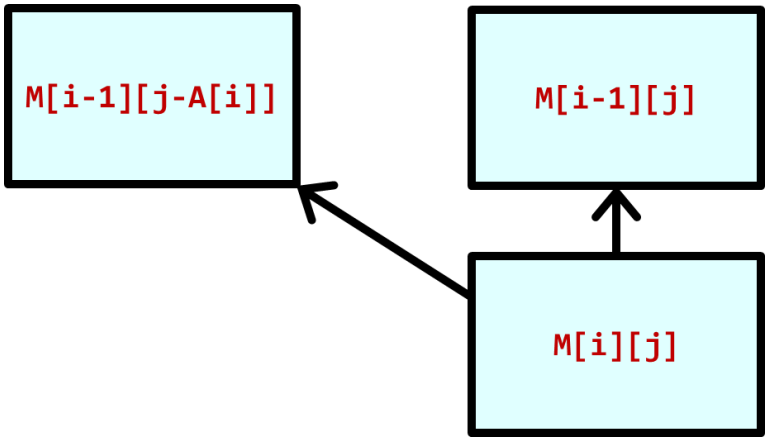

1,1
1,2
1,5
1,6
2,1
2,2
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6

e.g. A[4] = 2

Question 3

Ordering (I): Okay

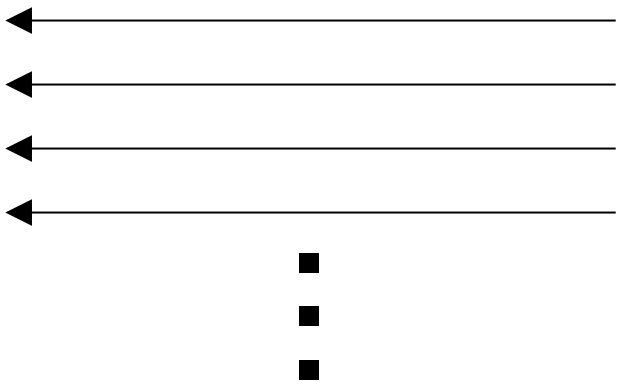


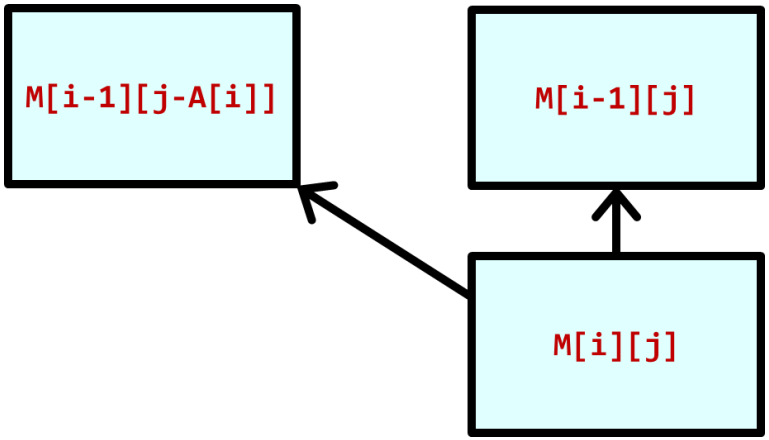



```
for i ← 1 to n
    for j ← 0 to total
```

Question 3

Ordering (II): Okay

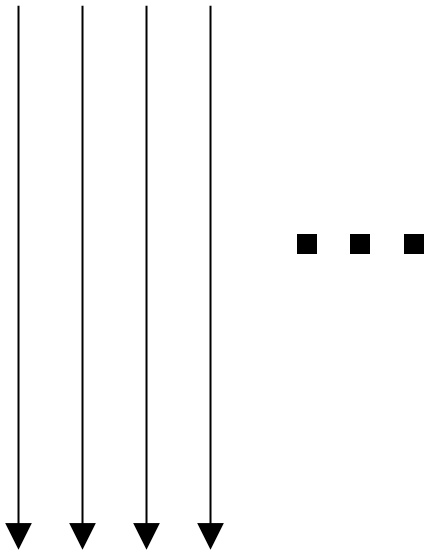




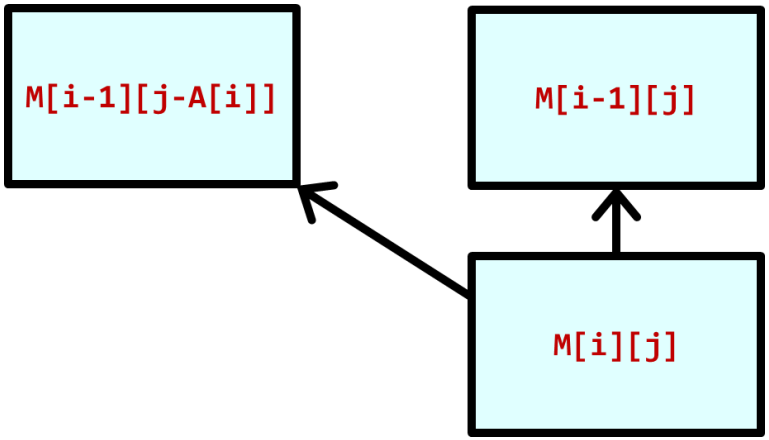

```
for i ← 1 to n
    for j ← total downto 0
```

Question 3

Ordering (III): Okay



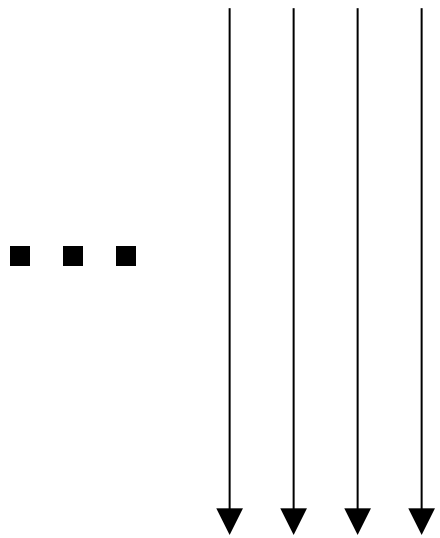
...

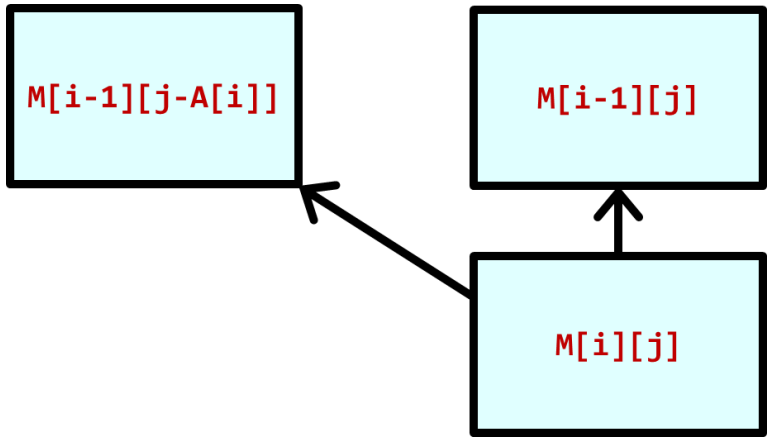


for j ← 0 to total
    for i ← 1 to n

Question 3

Ordering (IV): Wrong, table entries used before computed





for j ← total downto 0
    for i ← 1 to n

Question 4

PARTITION: Given an array of positive integers $A[1..n]$, can the elements in the array be partitioned into two sets of equal total sum?

Let S be the sum of elements in A. Which of the following statements regarding PARTITION are true?

(I) Memoization may solve fewer subproblems than bottom-up dynamic programming to obtain the final solution.

(II) It is possible to solve PARTITION using bottom-up dynamic programming in $O(nS)$ time using **only** $O(S)$ **space**.

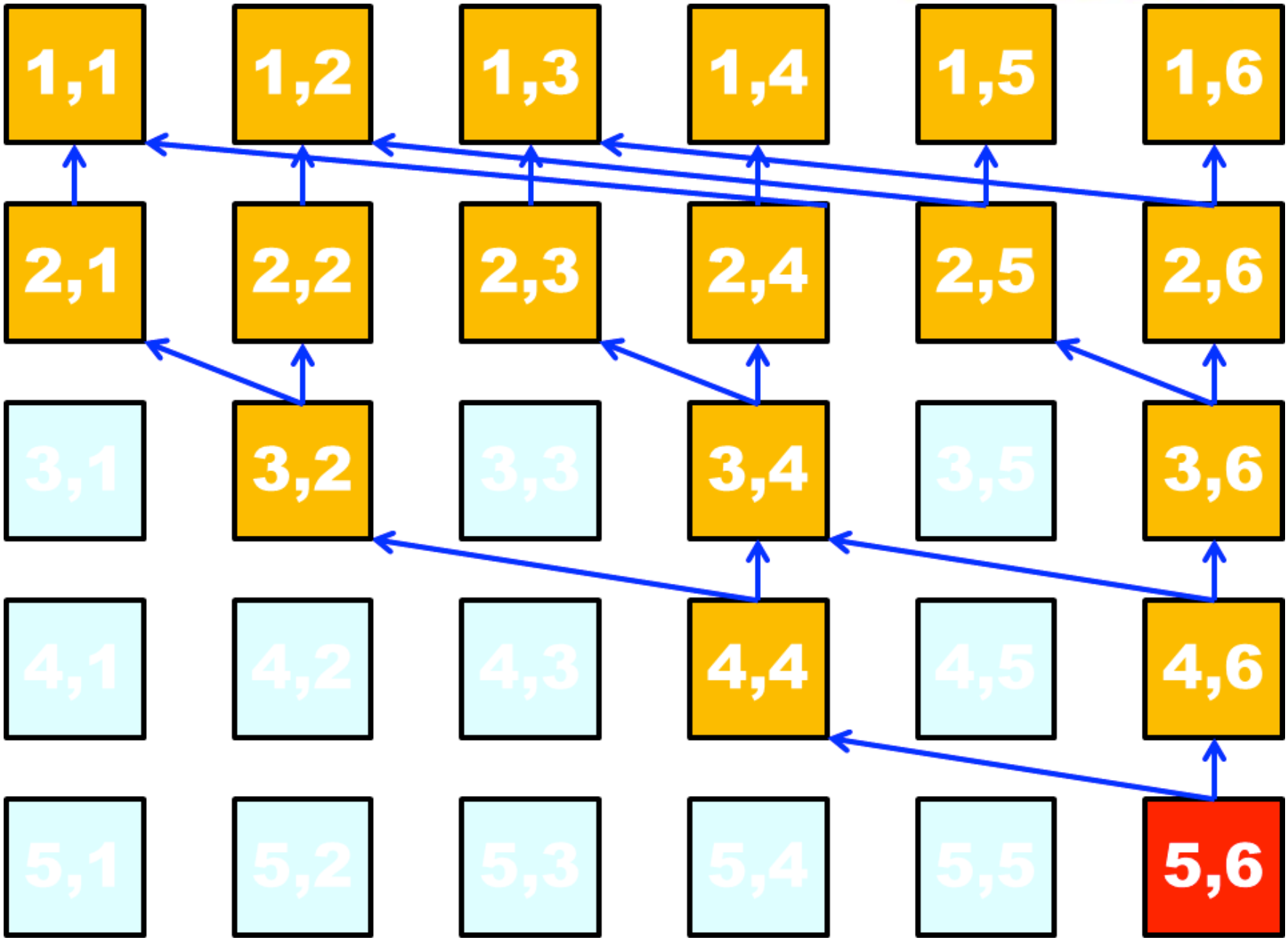
- None.
- (I) only
- (II) only
- Both (I) and (II)

Question 4

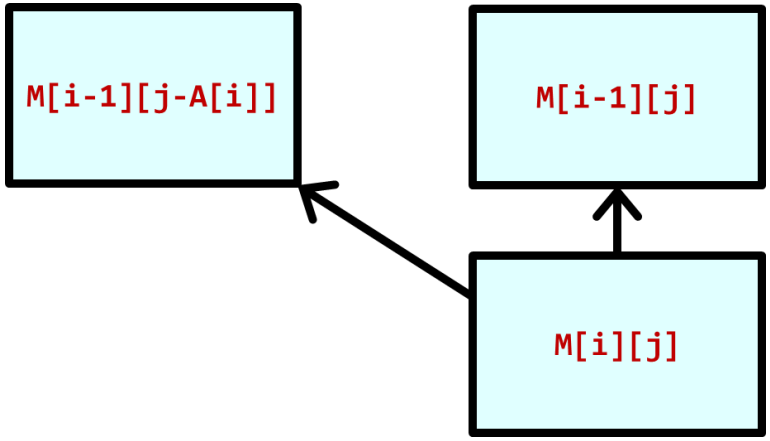
Answer: D Both (I) and (II)

For (I), memoization recursively calls (Partition(A,n-1,$total$) **or** Partition(A,n-1,$total$-A[n])).

- Only some subproblems are solved, e.g. does not solve for Partition(A,n-1,$total$-1) if A[n] is not equal to 1.


1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6

Question 4



Answer: D Both (I) and (II)

For (II): Use $O(S)$ space

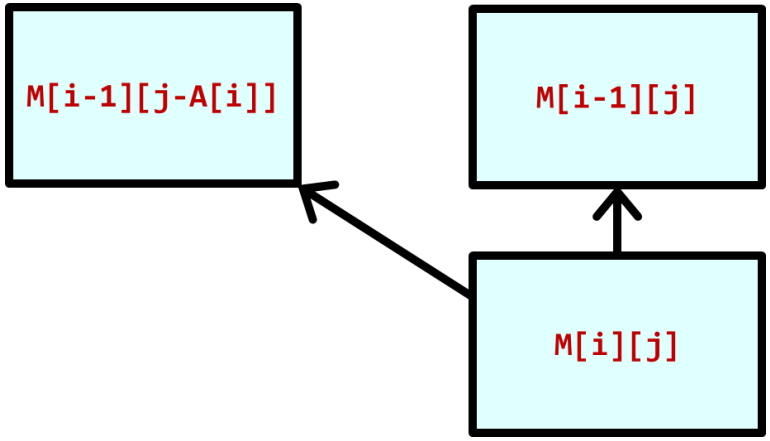
Note only previous row required to compute current row.

Store only two rows: *curr* and *prev*.

After finish computing *curr*, swap row with *prev*.

```
Partition(A,total)
for i ← 1 to n
    for j ← 0 to total
        if (j ≥ A[i]) then
            M[i][j] ← (M[i-1][j]
                  or M[i-1][j-A[i]])
        else
            M[i][j] ← M[i-1][j]
```

Question 4

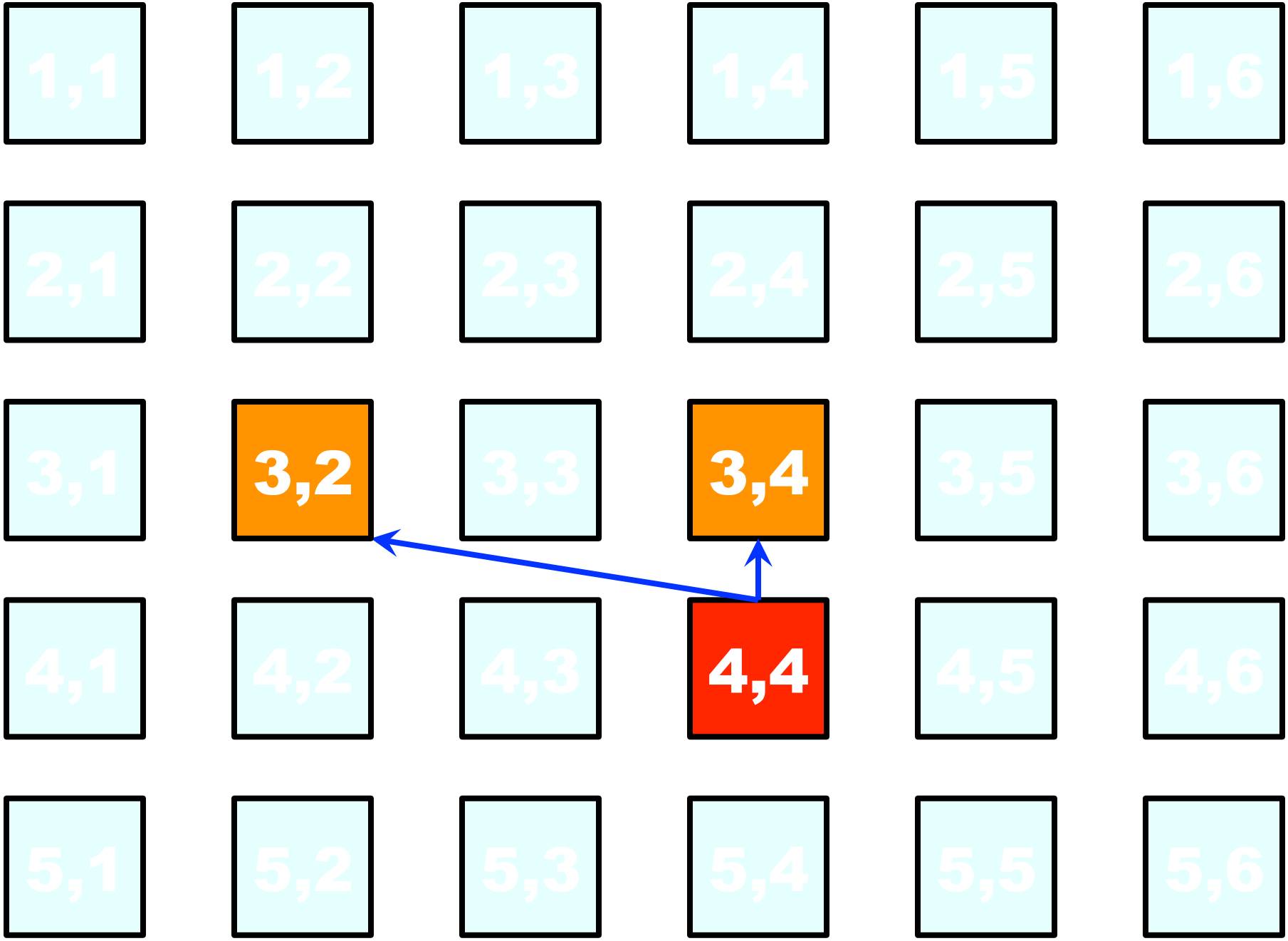


Answer: D Both (I) and (II)

For (II): Use $O(S)$ space

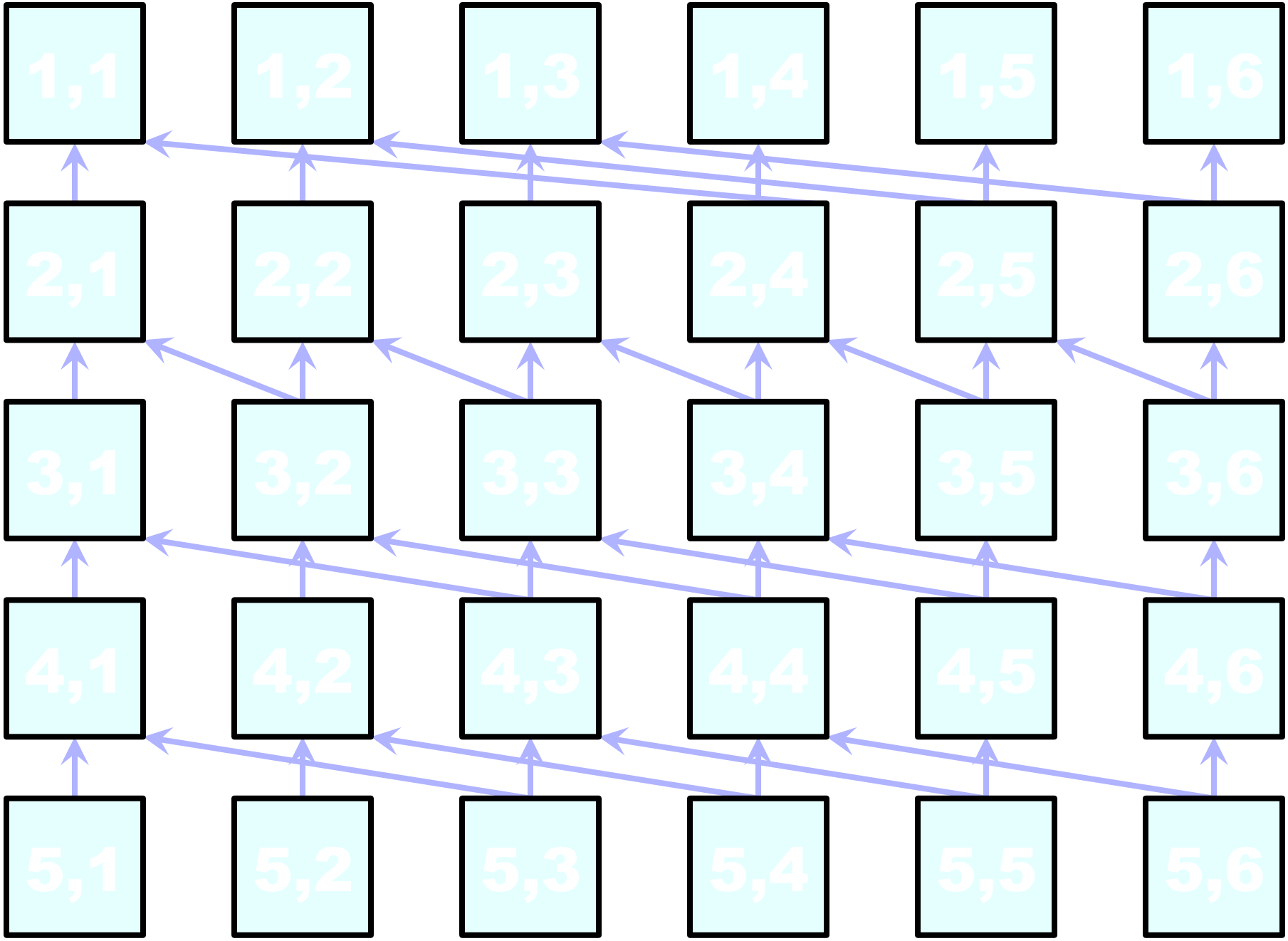
Even better, can use just one row only and overwrite it by iterating from *total* downto 0, as shown in code here

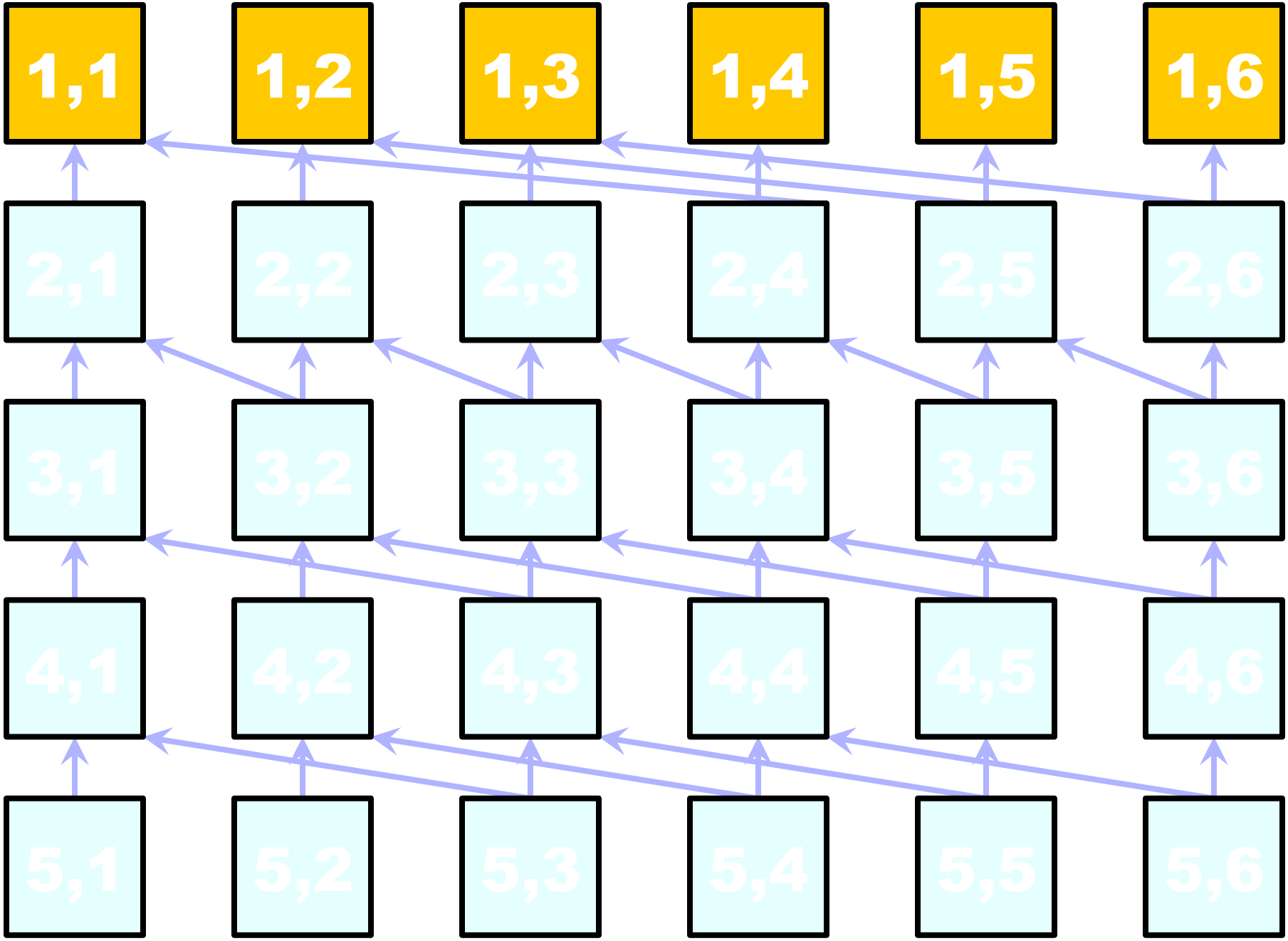
```
Partition(A,total)
for i ← 1 to n
    for j ← total downto 0
        if (j ≥ A[i]) then
            M[j] ← (M[j]
                    or M[j-A[i]])
        else
            M[j] ← M[j]
```

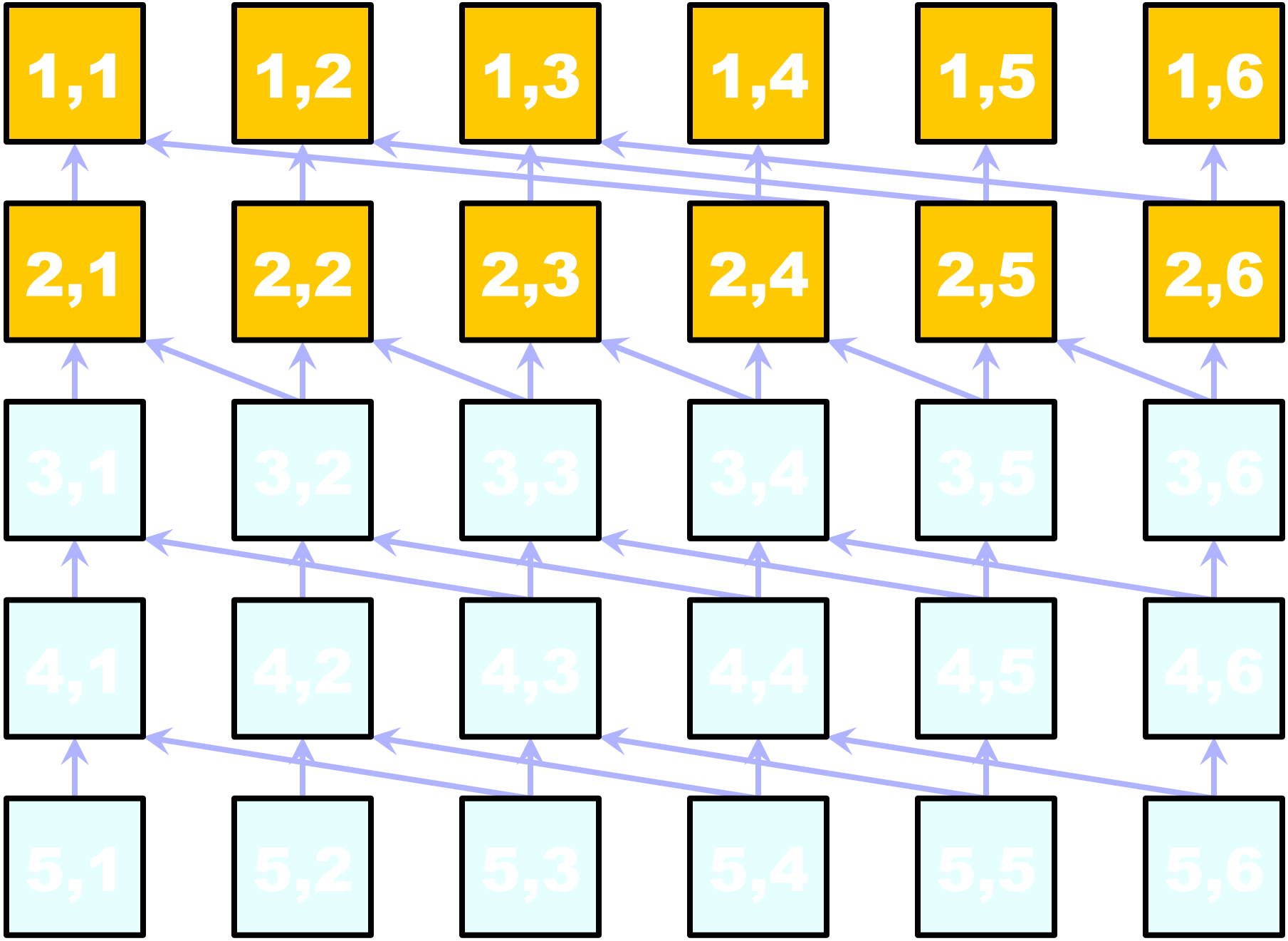

1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6

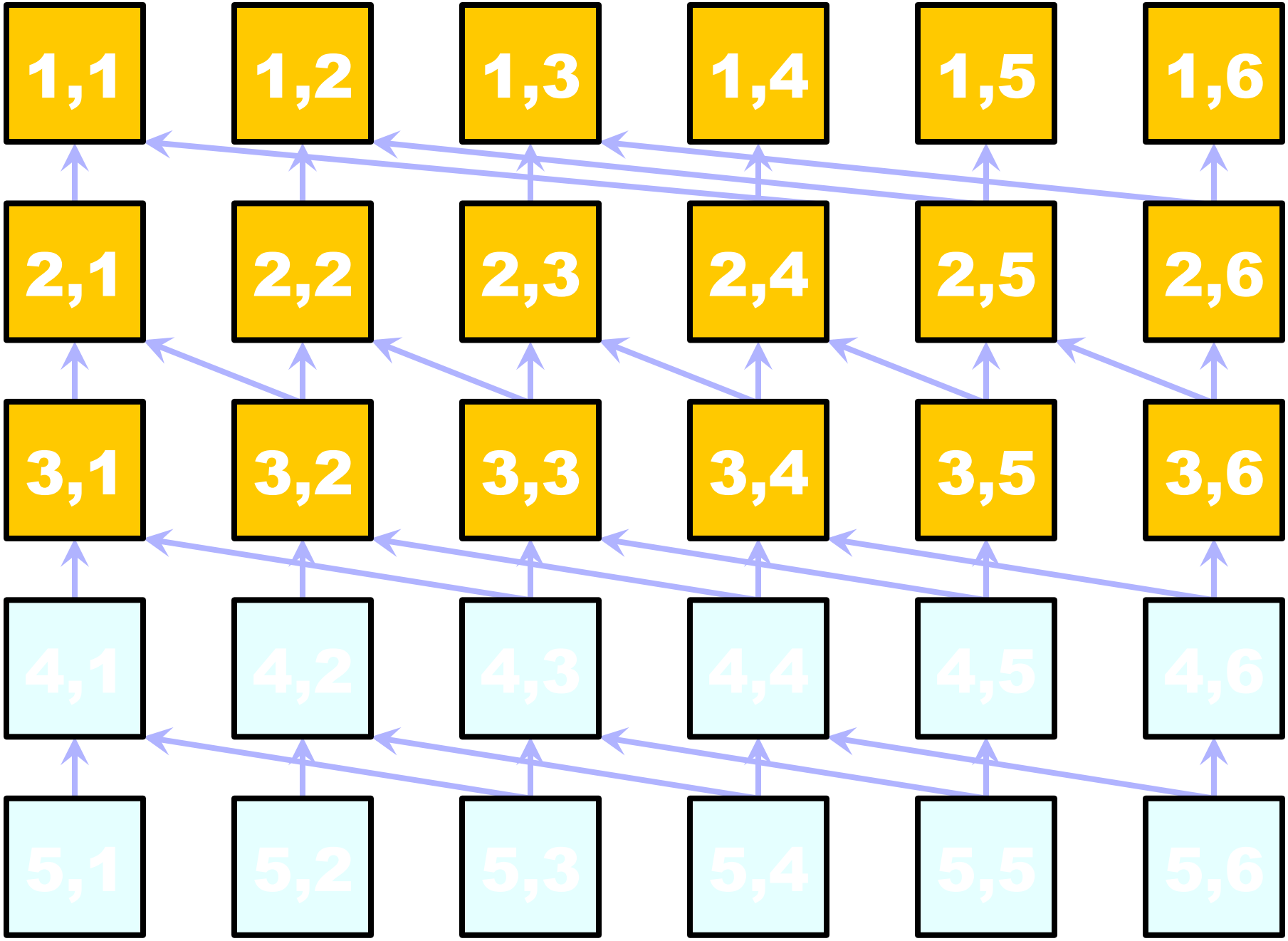
Question 4

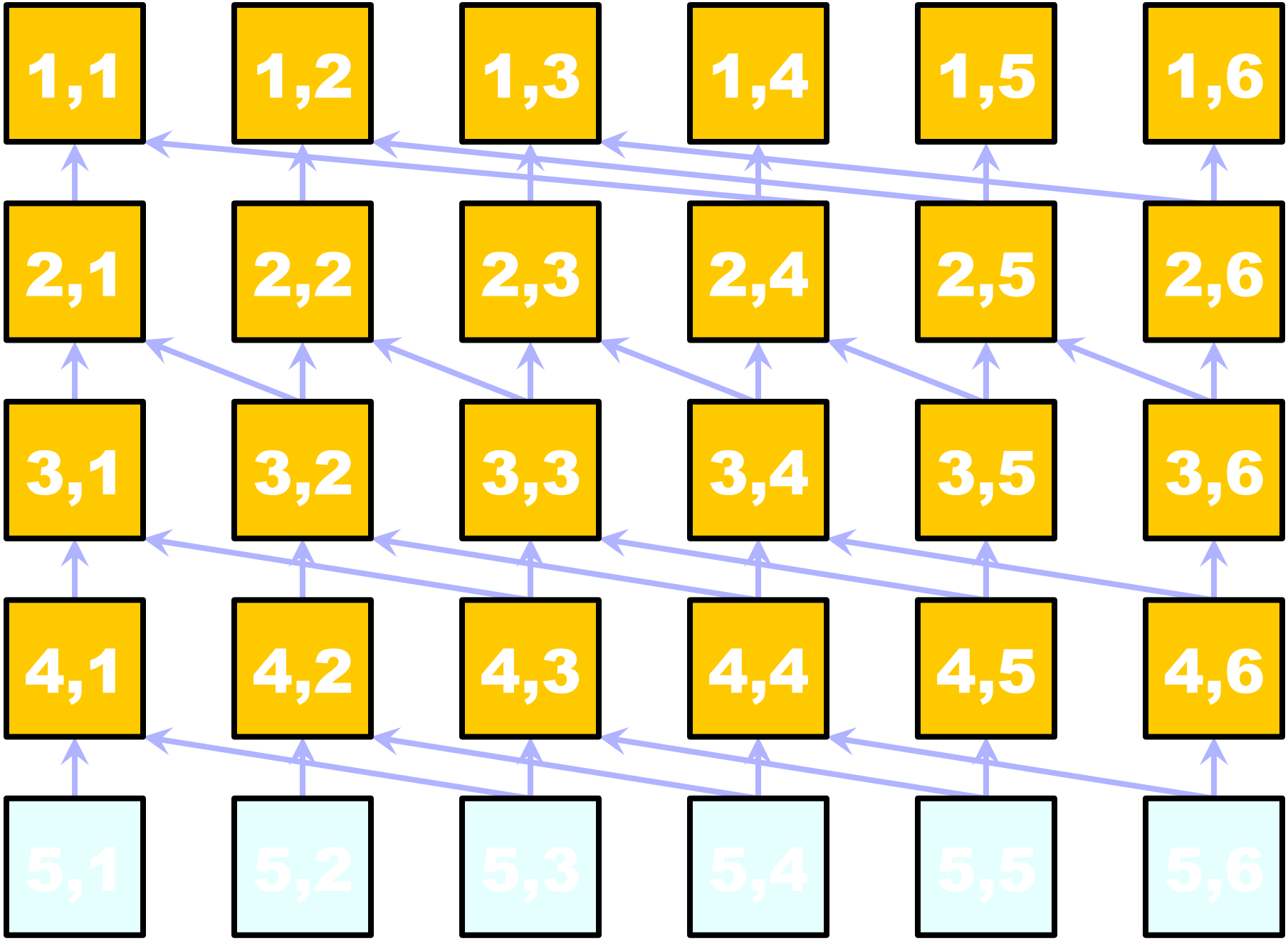
Bottom-Up DP for PARTITION: Original $\Theta(Sn)$ space

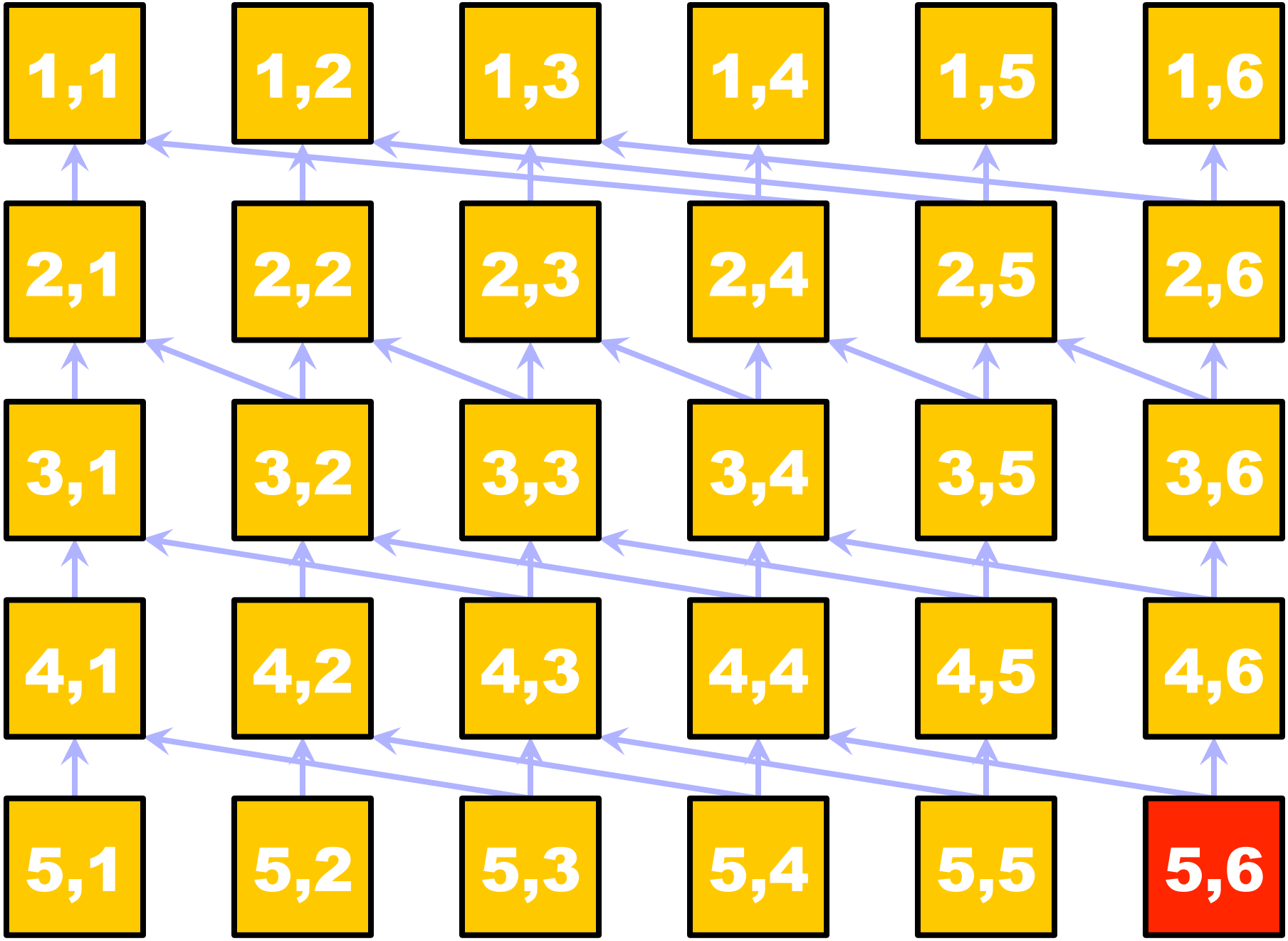

1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6


1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6


1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6


1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6


1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6


1,1
1,2
1,3
1,4
1,5
1,6
2,1
2,2
2,3
2,4
2,5
2,6
3,1
3,2
3,3
3,4
3,5
3,6
4,1
4,2
4,3
4,4
4,5
4,6
5,1
5,2
5,3
5,4
5,5
5,6

Question 4

Bottom-Up DP for PARTITION: 2 Rows $\Theta(S)$ space

1,1	1,2	1,3	1,4	1,5	1,6
2,1	2,2	2,3	2,4	2,5	2,6

1,1	1,2	1,3	1,4	1,5	1,6
2,1	2,2	2,3	2,4	2,5	2,6

1,1	1,2	1,3	1,4	1,5	1,6
2,1	2,2	2,3	2,4	2,5	2,6

3,1	3,2	3,3	3,4	3,5	3,6
2,1	2,2	2,3	2,4	2,5	2,6

3,1	3,2	3,3	3,4	3,5	3,6
2,1	2,2	2,3	2,4	2,5	2,6

3,1	3,2	3,3	3,4	3,5	3,6
4,1	4,2	4,3	4,4	4,5	4,6

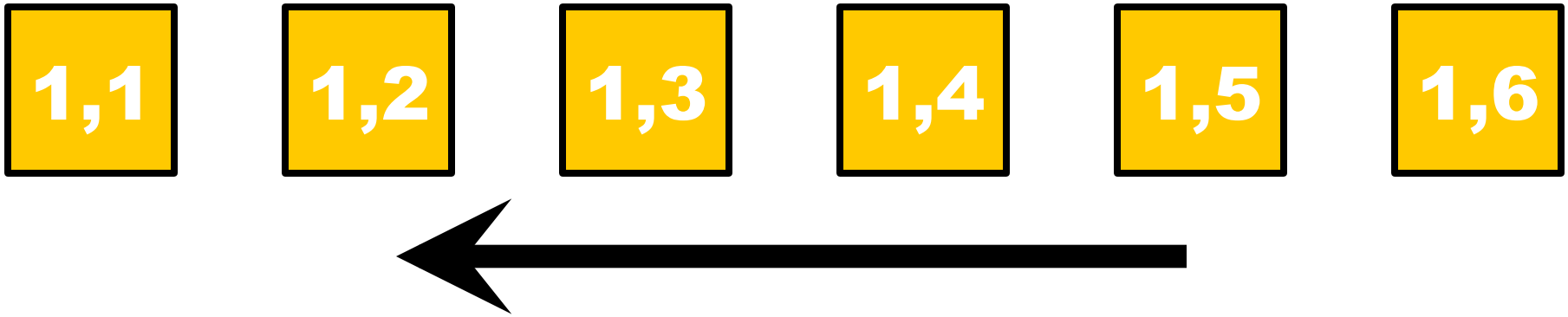
3,1	3,2	3,3	3,4	3,5	3,6
4,1	4,2	4,3	4,4	4,5	4,6

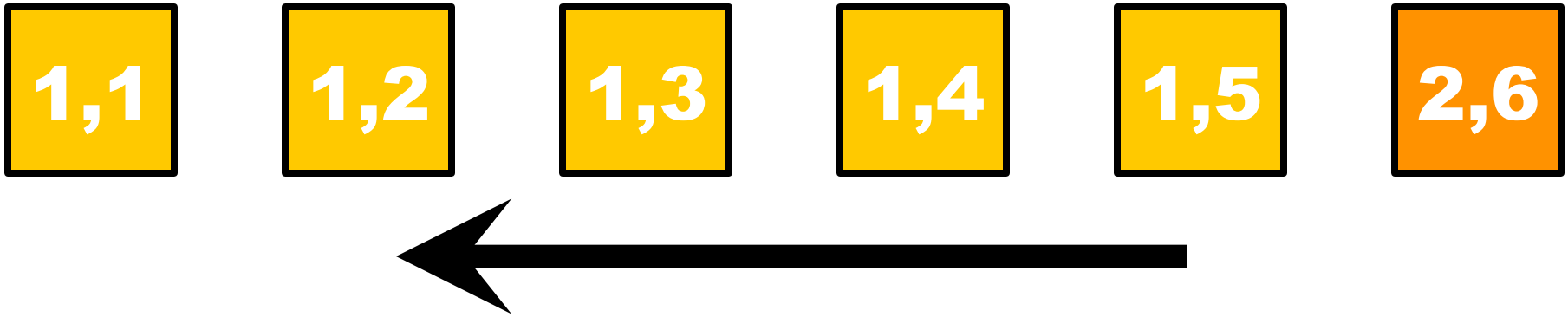
5,1	5,2	5,3	5,4	5,5	5,6
4,1	4,2	4,3	4,4	4,5	4,6

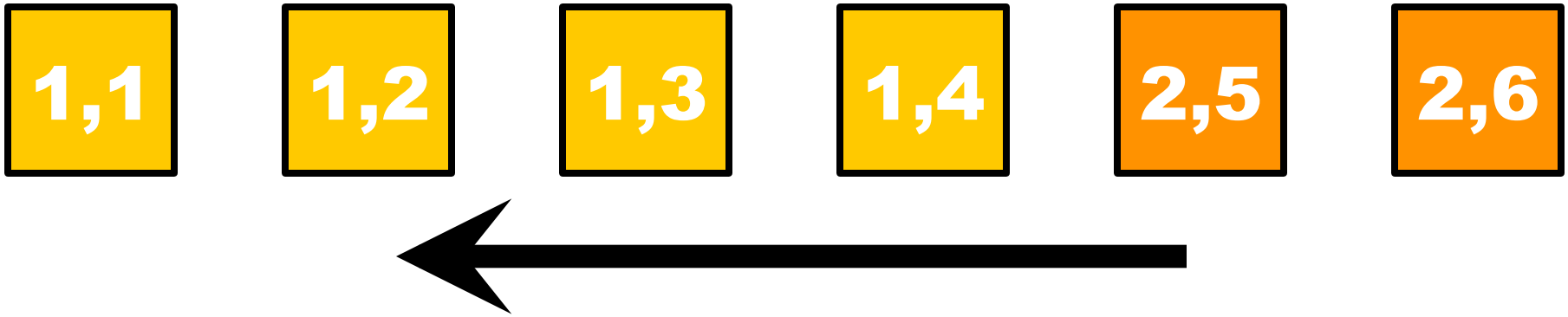
5,1	5,2	5,3	5,4	5,5	5,6
4,1	4,2	4,3	4,4	4,5	4,6

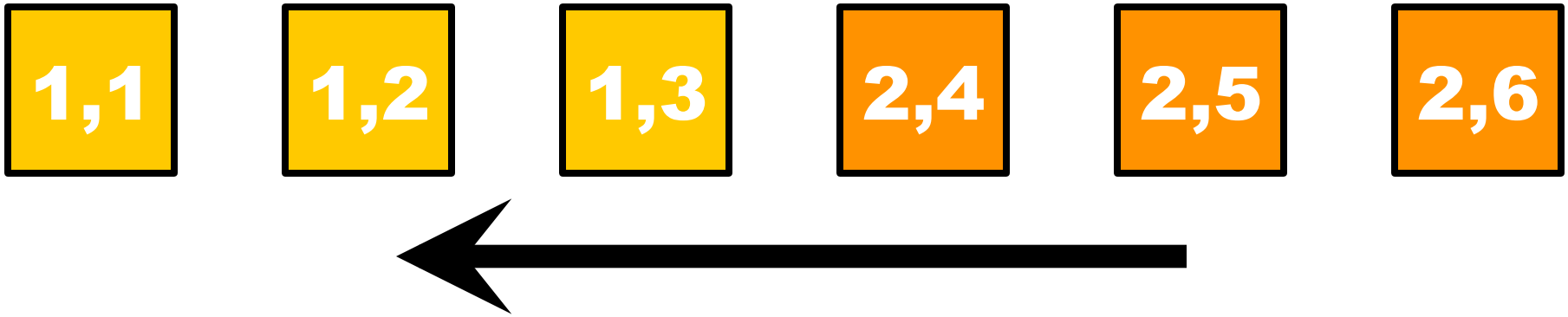
Question 4

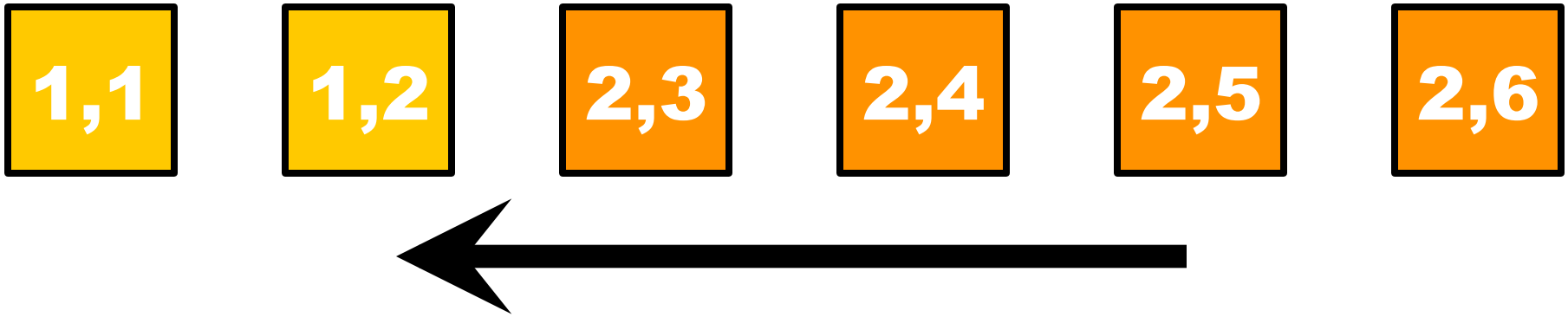
Bottom-Up DP for PARTITION: 1 Row $\Theta(S)$ space

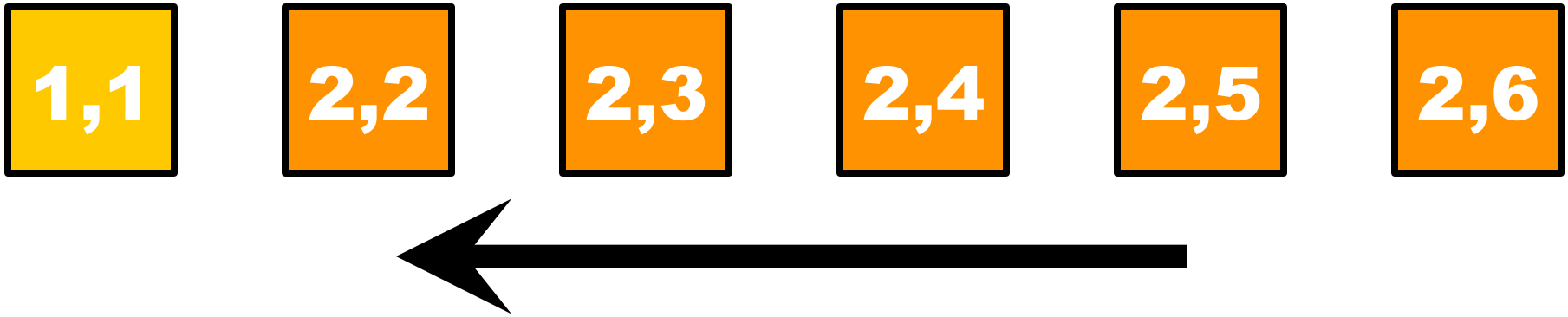

1,1
1,2
1,3
1,4
1,5
1,6

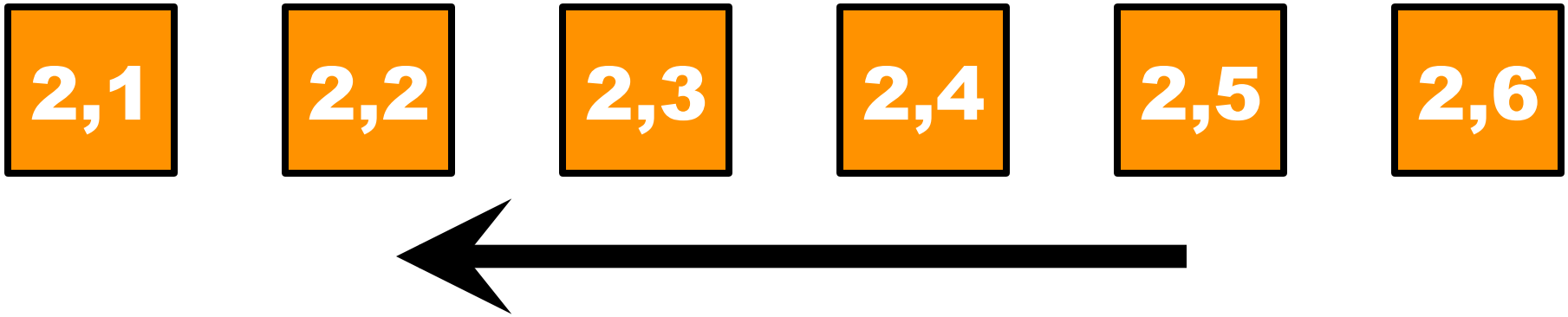

1,1
1,2
1,3
1,4
1,5
2,6


1,1
1,2
1,3
1,4
2,5
2,6


1,1
1,2
1,3
2,4
2,5
2,6


1,1
1,2
2,3
2,4
2,5
2,6


1,1
2,2
2,3
2,4
2,5
2,6


2,1
2,2
2,3
2,4
2,5
2,6